

本次实验包含使用分类和回归两个任务，step1 和 step2 为分类任务，step3 和 step4 为回归任务

Step1

使用天气数据集，手动实现 ID3 决策树模型，判断今天是否适合打网球

数据集介绍：

天气	温度	湿度	是否有风	是否适合打网球
晴	热	高	否	否
晴	热	高	是	否
阴	热	高	否	是
雨	温	高	否	是
雨	凉爽	中	否	是
雨	凉爽	中	是	否
阴	凉爽	中	是	是
晴	温	高	否	否
晴	凉爽	中	否	是
雨	温	中	否	是
晴	温	中	是	是
阴	温	高	是	是
阴	热	中	否	是
雨	温	高	是	否

要求：计算补全代码中的 `_calc_shannon_ent` 和 `_choose_best_feature` 函数

```
def _calc_shannon_ent(self, dataSet):
    """计算数据集的香农熵
    Args:
        dataSet: 数据集
    Returns:
        香农熵值
    """
    numEntries = len(dataSet)
    labelCounts = {} # 统计每个类别出现的次数
    for featVec in dataSet:
        currentLabel = featVec[-1]
        # TODO: 统计每个标签 currentLabel 出现的次数，存入 labelCounts 中
        pass

    entropy = 0.0
    for key in labelCounts:
```

```

        prob = ... # TODO: 计算某个标签的概率
        entropy -= ... # TODO: 累加计算香农熵, 使用 log(prob, 2)
    return entropy

def _choose_best_feature(self, dataSet):
    """选择最佳划分特征
    Args:
        dataSet: 数据集
    Returns:
        最佳特征的索引
    """
    numFeatures = len(dataSet[0]) - 1 # 特征数量
    baseEntropy = self._calc_shannon_ent(dataSet) # 原始熵
    bestInfoGain = 0.0
    bestFeature = -1
    for i in range(numFeatures):
        featList = [example[i] for example in dataSet]
        uniqueVals = set(featList) # 该特征的所有可能取值
        newEntropy = 0.0
        for value in uniqueVals:
            subDataSet = self._split_data_set(dataSet, i, value)

            prob = ... # TODO: 子数据集占总集的比例
            newEntropy += ... # TODO: 加权求和熵值
            infoGain = ... # TODO: 计算信息增益

        print("特征{}的信息增益: {:.3f}".format(i, infoGain))
        if infoGain > bestInfoGain:
            bestInfoGain = infoGain
            bestFeature = i
    return bestFeature

```

Step2

在蘑菇数据集上使用 Sklearn 决策树进行分类, 打印出关键指标, 观察分类结果

蘑菇数据集介绍: <https://www.selectdataset.com/dataset/bdd765e0d7aca471e9a5cd878eef794f>

要求 (可选): 对step1 中的天气数据集, 尝试使用 Sklearn中的决策树进行分类, 对比结果

```

# 决策树分类实验-蘑菇数据集
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay

```

```

)
import matplotlib.pyplot as plt

# 1. 加载蘑菇数据集
df = pd.read_csv("mushrooms.csv")
print("蘑菇数据集前5行: \n", df.head())

# 特征编码 (将类别型数据转为独热编码)
X = pd.get_dummies(df.drop('class', axis=1)) # 特征
y = df['class'].map({'e': 0, 'p': 1})      # 标签: edible->0, poisonous->1
class_names = ['edible', 'poisonous']

# 2. 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. 构建并训练决策树模型
clf = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)
clf.fit(X_train, y_train)

# 4. 模型预测与评估
y_pred = clf.predict(X_test)

# 准确率
acc = accuracy_score(y_test, y_pred)
print(f"测试集准确率: {acc:.4f}")

# 分类报告
print("\n分类报告 (Classification Report) : ")
print(classification_report(y_test, y_pred, target_names=class_names))

# 混淆矩阵
print("混淆矩阵 (Confusion Matrix) : ")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# 混淆矩阵可视化
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap=plt.cm.Greens)
plt.title("Confusion Matrix")
plt.show()

# 5. 决策树可视化 (只显示前2层避免图片过长)
plt.figure(figsize=(20, 12))
plot_tree(
    clf,
    feature_names=X.columns,
    class_names=class_names,
    filled=True,
    rounded=True,
    max_depth=2,

```

```
    fontsize=10
)
plt.title("Decision Tree (max_depth=2)")
plt.show()
```

Step3

手动实现回归树，对加州房价数据集进行预测

数据集介绍：<https://www.selectdataset.com/dataset/2cd2773c07369e0bdf2782e9831c2f32>

要求：补全代码

```
def mse(y):
    """计算目标值的均方误差 (Mean Squared Error)
    Args:
        y: 目标值数组，形状为(n_samples,)
    Returns:
        均方误差值
    """
    # TODO: 实现 MSE 计算公式
    return ...

def best_split_regression(X, y):
    """寻找最佳分割特征和阈值
    Args:
        X: 特征矩阵，形状为(n_samples, n_features)
        y: 目标值数组，形状为(n_samples,)
    Returns:
        best_feat: 最佳分割特征的索引
        best_thresh: 最佳分割阈值
    """
    m, n = X.shape
    best_loss = float("inf") # 初始化最小损失为无穷大
    best_feat, best_thresh = None, None
    for feat in range(n): # 遍历所有特征
        thresholds = np.unique(X[:, feat]) # 获取该特征所有唯一值作为候选阈值
        for thresh in thresholds:
            left = X[:, feat] <= thresh # 左子树样本掩码
            right = ~left # 右子树样本掩码
            if len(y[left]) == 0 or len(y[right]) == 0: # 跳过无效分割
                continue

            # TODO: 计算当前分裂的加权 MSE (作为分裂损失)
            loss = ...

            if loss < best_loss: # 更新最佳分割
                best_loss = loss
                best_feat = feat
```

```

        best_thresh = thresh
    return best_feat, best_thresh

def _build_tree(self, X, y, depth):
    """递归构建决策树（带预剪枝）
    Args:
        x: 当前节点的特征
        y: 当前节点的目标值
        depth: 当前深度
    Returns:
        叶子节点返回均值，内部节点返回(feat, thresh, left, right)元组
    """
    # 预剪枝条件
    if (len(y) < self.min_samples_split or # 样本数不足
        depth >= self.max_depth or # 达到最大深度
        len(set(y)) == 1): # 所有样本目标值相同
        return np.mean(y)

    feat, thresh = best_split_regression(X, y)
    if feat is None:
        return np.mean(y)

    left = X[:, feat] <= thresh
    right = ~left

    # 检查叶节点最小样本数
    if sum(left) < self.min_samples_leaf or sum(right) < self.min_samples_leaf:
        return np.mean(y)

    # TODO: 递归构造左右子树
    left_subtree = ...
    right_subtree = ...
    return (feat, thresh, left_subtree, right_subtree)

```

Step4

使用 Sklearn 随机森林进行房价预测，将结果与前面手动实现的回归树进行对比，说说你的理解

要求（可选）：自选一个其它合适数据集，使用 Sklearn 中的回归树进行预测

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 加载加州房价数据集
data = fetch_california_housing()

```

```

X, y = data.data, data.target
feature_names = data.feature_names
print("特征维度:", X.shape)
print("特征名称:", feature_names)

# 数据划分
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 初始化并训练随机森林模型
model = RandomForestRegressor(
    n_estimators=200, # 决策树数量
    max_depth=8, # 单棵树最大深度
    min_samples_split=5, # 节点分裂最小样本数
    min_samples_leaf=5, # 叶节点最小样本数
    max_features=0.8, # 每次分裂只考虑80%的特征
    random_state=42,
    n_jobs=-1 # 并行加速
)
model.fit(X_train, y_train)

# 模型预测
y_pred = model.predict(X_test)

# 评估指标
print("\n=== sklearn 随机森林评估 ===")
print("均方误差 (MSE):", mean_squared_error(y_test, y_pred))
print("R2 分数:", r2_score(y_test, y_pred))

# 特征重要性可视化
plt.figure(figsize=(10, 6))
importances = model.feature_importances_
indices = np.argsort(importances)[::-1]
plt.title("特征重要性")
plt.bar(range(X.shape[1]), importances[indices], color="forestgreen", align="center")
plt.xticks(range(X.shape[1]), np.array(feature_names)[indices], rotation=45)
plt.xlim([-1, X.shape[1]])
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.close()

# 可视化预测效果
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.6, color='forestgreen')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--')
plt.xlabel("实际房价")
plt.ylabel("预测房价")
plt.title("sklearn 随机森林拟合效果 (加州房价)")
plt.grid(True)
plt.tight_layout()
plt.show()

```

